

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

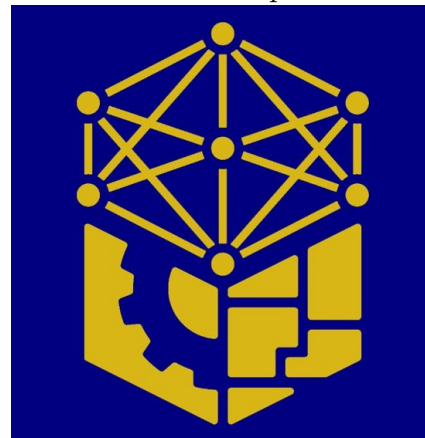
Polo Chura, Marco Rosau

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	iv
Índice de figuras	v
Introducción	1
1 Revision de S.O. existentes	4
1.1 Windows	5
1.2 Mac OS	7
1.3 Linux	9
1.4 MINIX 3	11
1.5 RedoxOs	13
1.6 xv6	15
1.7 FreeRTOS	17
1.8 Fuchsia	19
1.9 Haiku	21
1.10 ToaruOS	22
2 Comparación técnica	24

Índice de tablas

2.1	Comparación resumida de distintos sistemas operativos en orden de popularidad, elaborada en base a la investigación realizada en este proyecto.	25
-----	---	----

Índice de figuras

1.1	Ícono de Windows	5
1.2	Ícono de macOS	7
1.3	Ícono de Linux	9
1.4	Ícono de MINIX 3	11
1.5	Ícono de RedoxOS	13
1.6	Ícono de xv6	15
1.7	Ícono de FreeRTOS	17
1.8	Arquitectura de FreeRTOS	18
1.9	Ícono de FuchsiaOS	19
1.10	Ícono de HaikuOS	21
1.11	Ícono de ToaruOS	22

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Revision de S.O. existentes

1.1 Windows



Figura 1.1: Ícono de Windows (Wikimedia Commons contributors, 2021)

Windows es un sistema operativo desarrollado por Microsoft, cuyo origen se remonta a 1985 con la versión Windows 1.0, como una interfaz gráfica que extendía las funciones de MS-DOS. Desde entonces ha evolucionado hasta convertirse en una plataforma dominante en PCs, con múltiples versiones comerciales orientadas a usuarios domésticos, negocios y servidores (Alyssa Marie Rafael Natividad, 2023).

Windows posee una arquitectura monolítica, ya que gran parte del núcleo y los controladores E/S del dispositivo se procesan en modo kernel compartiendo la memoria. Windows es similar al sistema de UNIX, Windows tiene incluido una abstracción de hardware (HAL) y subsistema en modo usuario con el Win 32, POSIX, OS/2. (Mark E. Russinovich and David A. Solomon, 2005)

Está desarrollado en el lenguaje de programación C con sus componentes de bajo nivel como gestor de memoria, procesos, administración de E/S y partes de la interfaz del usuario y del sistema se implementaron en C++ y en ensamblador para las rutinas más sensibles, como arranque, gestión de interrupciones y entre otros (Mark E. Russinovich and David A. Solomon, 2005).

Entre sus componentes clave se encuentran el planificador de procesos y subprocesos que maneja procesos e hilos en modo kernel, el gestor de memoria virtual con paginación que protege cada proceso, subsistemas de archivos como NTFS, soporte de

red integral, sistema de drivers para dispositivos hardware, interfaz gráfica de usuario (GUI), componentes de seguridad como control de acceso, y el HAL para manejar diferencias entre arquitecturas de hardware (Justin E. Forrester and Barton P. Miller, 2000).

Windows tiene una comunidad muy grande de usuarios, desarrolladores y empresas. La documentación se publica mediante Microsoft Docs, los libros “Windows Internals” son referencias reconocidas, y Microsoft suele liberar guías técnicas, SDKs y APIs para desarrolladores. Sin embargo, el código fuente no es completamente abierto, lo que limita análisis y ediciones o posibles cambios en el código.

1.2 Mac OS



Figura 1.2: Ícono de macOS (GQ Informática, 2023)

El proyecto Mach dio comienzo en 1985 en la universidad de Carnegie Mellon con la finalidad de crear un microkernel que reemplace al BSD. Su diseño buscaba más seguridad para los usuarios, nose genero lo resultados esperados y se cancelo en 1994. Entonces el proyecto fue adoptado pro OSF/1 Y NeXTSTEP por lo que completo al combinar los componente de BSD, dando origen al kernel XNU de tipo monolítico, pero presento limitaciones con el System 7 y fracaso el proyecto, por ultimo este último avance fue adquirida por Steve Jobs, lo que permitió integrar NeXTSTEP y su kernel XNU como nuevo sistema operativo a Mac OS X, lanzado en 24 marzo del 2001 con el kernel de XNU, que integraba Mach 3.0 y el marco I/O kit, además de ser integrado en Intel Y ARM (Daan Keuper, 2012). El kernel de XNU prosee una arquitectura hibrida que tiene 4 componentes clave **Mach** que se encarga de administrar las tares, que contine hilos. Su característica principal es la mensajería, mediante puntos de comunicación (IPC), usa el Mach Interface Generator (MIG) para acortar lo procesos, tenemos tambien **BDS** que implementa procesos y señales UNIX, adema de sistema de archivos y redes, **I/O Kit** es el marco de controladores orientado a objetos y por ultimo **KEXTs** que encargar de enlazar no solo con el I/O Kit, sino tambien con

otras partes del kernel, extensión de red, sistema de archivos (Jonathan Levin, 2007). Varios de los primeros trabajos de software abierto de Apple son Darwin y WebKit. Darwin es un sistema operativo público que Apple introdujo en el año 2000. Este sistema fue liberado bajo la Licencia Pública de Apple (APSL), que ha sido aceptada por la OSI y la FSF (Jonathan Levin, 2007)

1.3 Linux



Figura 1.3: Ícono de Linux (Wikipedia, 2022)

Linux es un sistema operativo de tipo Unix iniciado en 1991 por Linus Torvalds como un kernel monolítico, modular y multitarea. Originalmente escrito para PCs i386, ha evolucionado hasta ofrecer compatibilidad POSIX y soporte extenso de hardware, siendo desarrollado bajo el modelo de código abierto, donde las mejoras provienen de contribuyentes individuales y corporativos, mientras que la dirección general la marca la comunidad y no un único proveedor (Wikipedia, 2022; Oracle latam, 2024). Su arquitectura se basa en un kernel monolítico moderno que integra características modulares. Todo el núcleo, incluidos los controladores de dispositivos, se ejecuta en un único espacio de direcciones en modo kernel. Sin embargo, soporta la carga y descarga dinámica de módulos en tiempo de ejecución, lo que le otorga flexibilidad similar a la de los micronúcleos sin sacrificar el rendimiento propio de un diseño monolítico (Robert Love, 2010, pp. 7). El núcleo de Linux está escrito principalmente en C, con secciones críticas en ensamblador para optimizar la interacción directa con el hardware. Las aplicaciones en espacio de usuario, en cambio, pueden desarrollarse en diversos lenguajes como C, C++, Python o Rust.

Entre sus componentes principales se encuentran la gestión de memoria con asignación y protección, la planificación de procesos que regula el uso de la CPU, los controladores de dispositivos para el hardware, el sistema de llamadas al sistema con sus mecanismos de seguridad, y el sistema de archivos virtual (VFS), encargado de unificar múltiples formatos y módulos adicionales (Oracle latam, 2024).

Linux posee una de las comunidades de código abierto más amplias y activas del mundo, integrada por miles de desarrolladores pertenecientes a empresas como Red Hat, Google, Intel o IBM, así como por colaboradores independientes. La documentación es igualmente extensa e incluye las **man pages**, el proyecto de documentación oficial del kernel (<https://docs.kernel.org/>), wikis como la Arch Wiki, foros especializados y numerosos libros técnicos.

1.4 MINIX 3



Figura 1.4: Ícono de MINIX 3 (Usenix, 2010)

MINIX fue creado en 1987 por Andrew Tanenbaum como herramienta docente para la enseñanza de sistemas operativos. En 2005 se anunció MINIX 3, una reimplementación centrada en la fiabilidad y la modularidad, cuyo propósito principal es ofrecer un sistema operativo compatible con POSIX, recuperable ante fallos y adecuado tanto para aplicaciones críticas como para entornos educativos. A diferencia de otros sistemas, fue escrito completamente desde cero, sin incluir código de AT&T, manteniendo compatibilidad con la versión 7 de Unix y los estándares POSIX (Usenix, 2010, pp. 10).

Su arquitectura está basada en un microkernel muy reducido que se encarga únicamente de interrupciones, gestión básica de procesos y comunicación mediante IPC. Todo lo demás se ejecuta en el espacio de usuario como servidores independientes: controladores de dispositivos, gestor de archivos, gestor de procesos y gestor de memoria. Este diseño multiserver permite que cada componente esté aislado, de modo que un fallo en un servidor no comprometa al sistema completo. Incluso cuenta con un servidor de reencarnación que reinicia automáticamente los componentes de-

fectuosos para mejorar la tolerancia a fallos (Jorrit N. Herder and Herbert Bos and Jakob Lichtenberg and Peter M. Chen and Andrew S. Tanenbaum, 2007).

MINIX 3 está escrito principalmente en C, con secciones en ensamblador dedicadas a la inicialización del hardware y a rutinas críticas de bajo nivel.

Entre sus componentes principales se encuentran la gestión de procesos, a cargo de un proceso supervisor que controla creación, terminación y recolección de fallos; la gestión de memoria, donde el microkernel provee segmentación y paginación básica mientras un servidor en espacio de usuario administra la memoria virtual; el servidor de archivos (Mini-FS) con soporte POSIX; controladores de dispositivos que se ejecutan como procesos de usuario aislados; y soporte para interfaces gráficas como X11 o entornos ligeros tipo EDE, siguiendo la tradición Unix (Usenix, 2010, pp. 12).

La comunidad de MINIX 3, aunque más pequeña en comparación con otros proyectos de código abierto, ha tenido impacto en el ámbito académico. Hasta 2010 su web oficial registraba cerca de 1.7 millones de visitas y más de 300,000 descargas de CD¹. El proyecto participó en Google Summer of Code entre 2008 y 2010, y mantiene recursos como wiki, grupos de discusión y repositorios en GitHub. Su documentación se apoya en el libro *Operating Systems: Design and Implementation* de Tanenbaum (3ª edición), artículos académicos y materiales de congresos, lo que lo convierte en una referencia educativa sólida en el campo de los sistemas operativos.

¹usenix.org

1.5 RedoxOs



Figura 1.5: Ícono de RedoxOS (Wikipedia contributors, 2025)

Redox es un sistema operativo tipo Unix que está escrito en el lenguaje de programación Rust, con el objetivo de implementar un microkernel y un sistema de aplicaciones. Rust es un lenguaje enfocado en la seguridad, rendimiento, gratuito y fácil de navegar. Redox se enfocó en la mejora de varios sistemas anteriores que presentaban errores. Uno de sus proyectos es Redox OS. Es compatible con POSIX, y la comunidad está trabajando para escribir la biblioteca libc en Rust, llamada relibc (Rohit Saini, 2018).

Dado que para los sistemas operativos es muy importante la seguridad, porque los sistemas operativos tienen un alto nivel de abstracción sobre los recursos del sistema, más aún en el caso de que Linux tuviera muchos errores en diferentes bibliotecas ocasionados por la seguridad de memoria. Rust en cambio evita todos esos problemas al tener seguridad de memoria en tiempo de compilación. El kernel de Redox contiene más de 20 mil líneas de código, y su diseño es de alto nivel por eso aún puede tener problemas (Simon Ellmann and Paul Emmerich, 2019).

El sistema operativo Redox OS se apoya en un microkernel y está inspirado en el sistema operativo MINIX. Las funciones tradicionales se implementan en kernels monolíticos, como controladores de dispositivo, pilas y el sistema de archivos. Los

microkernels son más seguros y ofrecen mayor estabilidad, pero una de las principales desventajas es que el rendimiento que ofrecen no está optimizado para la mayoría del hardware actual (Tomáš Ritter, 2019).

En cuanto a sus componentes clave, Redox OS integra un microkernel responsable de los procesos del sistema, un **sistema de archivos** implementado en espacio de usuario, **drivers** que también se ejecutan en espacio de usuario para incrementar la estabilidad, y compatibilidad POSIX que permite correr aplicaciones de otros sistemas Unix. También, añade **la biblioteca relibc**, escrita en Rust para mejorar la seguridad y la compatibilidad, además de **gestión de memoria** segura que evita vulnerabilidades comunes como los desbordamientos (The Redox OS Community, 2025).

1.6 xv6



Figura 1.6: Ícono de xv6 (Frans Kaashoek and Robert Morris, 2016)

xv6 es un sistema operativo educativo inspirado en Unix V6, desarrollado en 2006 por el MIT para su curso de sistemas operativos. Su propósito fue proporcionar un kernel moderno y simple que sustituyera al Unix V6 original, cuya complejidad dificultaba la enseñanza. xv6 sigue de cerca la estructura de Unix V6, pero está escrito en C ANSI y se ejecuta en arquitecturas Intel multiprocesador, convirtiéndose en una herramienta práctica y comprensible para la docencia (Frans Kaashoek and Robert Morris, 2016).

Su arquitectura corresponde a un kernel monolítico minimalista, en el que todo el núcleo del sistema operativo se ejecuta con privilegios de kernel, sin separación en servidores independientes. xv6 incluye soporte para multiprocesadores mediante bloqueos y estructuras internas de hilos, aunque carece de controladores de red o capacidades gráficas avanzadas. Sus componentes básicos abarcan la planificación de procesos con un esquema round-robin, la gestión de memoria con paginación simple, un sistema de archivos inspirado en Unix V6 y un conjunto reducido de llamadas al

sistema (Russ Cox and Frans Kaashoek and Robert Morris, 2022, pp. 25).

La implementación de xv6 está escrita principalmente en C, mientras que el código de inicio y las rutinas de interrupción se desarrollan en ensamblador x86. En sus versiones más recientes, también se encuentra una adaptación para la arquitectura RISC-V, lo que amplía su aplicabilidad en contextos educativos.

Entre los componentes principales destacan la gestión de procesos, que otorga a cada uno su propio espacio de direcciones, con un planificador round-robin y un cambio de contexto sencillo; la gestión de memoria mediante paginación básica y un asignador simple para el kernel; un sistema de archivos con inodos y directorios inspirado en Unix V6; y mecanismos elementales de sincronización como `sleep` y `wakeup`, que permiten estudiar las condiciones de carrera. Además, xv6 expone llamadas al sistema tradicionales de UNIX como `fork`, `exit`, `wait`, `open`, `read`, `write` y `close`, lo que facilita comprender la implementación exacta de estas primitivas fundamentales (Russ Cox and Frans Kaashoek and Robert Morris, 2022, pp. 26–30).

xv6 se ha consolidado como una herramienta académica ampliamente utilizada en universidades para cursos de sistemas operativos. Su código fuente está disponible públicamente, y los propios autores distribuyen un libro titulado *xv6: a simple, Unix-like teaching operating system*, que comenta línea por línea el código y explica tanto el funcionamiento como el diseño de sus estructuras y funciones. Gracias a su claridad y abundante documentación, xv6 se ha convertido en uno de los sistemas operativos de referencia para la enseñanza de fundamentos prácticos de diseño de kernels (Russ Cox and Frans Kaashoek and Robert Morris, 2022).

1.7 FreeRTOS



Figura 1.7: Ícono de FreeRTOS (Luis Llamas, 2025)

FreeRTOS es un sistema operativo en tiempo real con un kernel planificador, desarrollado para ejecutarse con microcontroladores. Está compuesto funcionalidades de planificación en tiempo real, comunicación entre tareas, análisis temporal y primitivas de sincronización con el propósito de cumplir con las tareas con el plazos estrictos de ejecución (Nannan He and Han-Way Huang, 2020). En cuanto a su arquitectura hace uso de un microkernel para administrar tareas en tiempo real, casi similar al RTLinux en que admiten núcleo dual, lo que permita separar tareas critica y no criticas (Anthony Serino and Liang Cheng, 2019) Esta escrito en lenguaje C estándar y con algunas lineas de código en ensamblador para que se acople a diferentes arquitecturas. Dentro de su componentes claves tenemos el **planificador**, **mecanismos de comunicación**, **sincronización** gracias al uso de colas y semáforos. Incluyendo la estructura de **Task Control Block (TCB)** para la administración de procesos y tareas (Richard Barry, 2018).

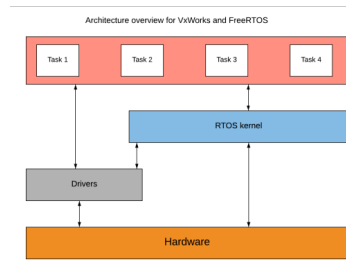


Figura 1.8: Diseño de la arquitectura de FreeRTOS (Luis Llamas, 2025)

FreeRTOS es código abierto con una licencia GPL, lo que permite su uso en aplicaciones comerciales. El código está disponible públicamente y cuenta con documentación extensa en el código fuente en su sitio oficial (The FreeRTOS Project, 2025).

1.8 Fuchsia



Figura 1.9: Ícono de FuchsiaOS (Fuchsia Project, 2025)

Fuchsia OS es un sistema operativo desarrollado por Google desde 2016, diseñado como una plataforma de propósito general para dispositivos embebidos, móviles, IoT y potencialmente de escritorio. A diferencia de Android, no se basa en Linux, sino en un núcleo propio denominado Zircon (Google Fuchsia Team, 2025b).

Su arquitectura está basada en un microkernel, en el que Zircon gestiona procesos, hilos, memoria y comunicación mediante objetos e IPC, mientras que servicios como sistemas de archivos y controladores se ejecutan en espacio de usuario, favoreciendo la seguridad y modularidad (Google Fuchsia Team, 2025a).

Fuchsia está escrito principalmente en C++, Rust y Dart. Zircon se implementa en C/C++, mientras que la capa de interfaz gráfica se desarrolla con Flutter (Dart), lo que facilita la portabilidad a múltiples plataformas (Google Inc., 2024).

Entre sus componentes clave se encuentran la gestión de procesos basada en objetos, memoria virtual con aislamiento estricto, soporte para múltiples sistemas de archivos (como MemFS, BlobFS y FAT) y un entorno gráfico con Flutter para experiencias multiplataforma (Google Fuchsia Team, 2025b).

El proyecto es mantenido principalmente por Google, con participación de la comunidad de código abierto. Su documentación oficial está disponible en [Fuchsia.dev](https://fuchsia.dev), que incluye guías técnicas, API y manuales de contribución, aunque parte del desarrollo se mantiene cerrado (Fuchsia Open Source Community, 2025).

1.9 Haiku



Figura 1.10: Ícono de HaikuOS(Haiku Project, 2025b)

Haiku es un sistema operativo de código abierto inspirado en BeOS, iniciado en 2001 con el objetivo de proporcionar un entorno moderno, rápido y sencillo de usar, orientado principalmente a equipos de escritorio (Haiku Project, 2025a).

Su arquitectura es híbrida, ya que utiliza un núcleo monolítico modular, pero con elementos que recuerdan a un microkernel, lo que equilibra rendimiento y flexibilidad (Haiku Project Developers, 2024). El sistema está desarrollado principalmente en C++, con partes en C y ensamblador para las interacciones de bajo nivel (Haiku Project Developers, 2024).

Entre sus componentes clave se incluyen un modelo de multitarea preventiva con planificación por prioridades, gestión de memoria virtual con paginación y aislamiento de procesos, y un sistema de archivos propio llamado OpenBFS, optimizado para indexación rápida (Haiku Project, 2023). Además, cuenta con una interfaz gráfica uniforme gestionada por el `app_server`.

Haiku es mantenido por la Haiku Project Foundation y su comunidad de desarrolladores voluntarios. El proyecto dispone de documentación oficial, foros y repositorios activos en GitHub, con lanzamientos beta periódicos que permiten probar y evaluar su madurez (Haiku Project Community, 2025).

1.10 ToaruOS

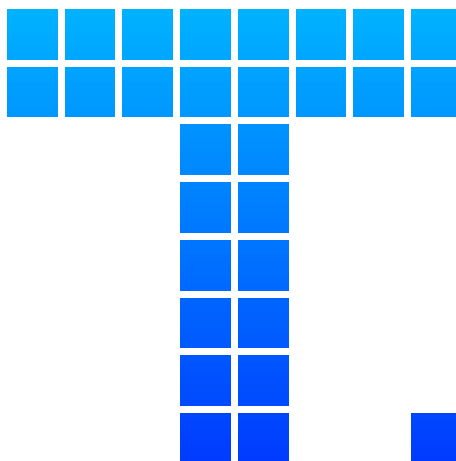


Figura 1.11: Ícono de ToaruOS (ToaruOS Project, 2025)

ToaruOS creado en 2010 por Kevin Lange, es un sistema operativo experimental con fines educativos, que busca evidenciar cómo se puede desarrollar un entorno completo desde la nada, independiente de kernels o librerías existentes. Su intención primigenia fue servir como soporte de investigación y aprendizaje para la comprensión del formato y funcionamiento interno de un sistema operativo moderno, incorporando subsistemas gráficos, de red y de usuario. A diferencia de proyectos como Linux o BSD, ToaruOS busca ser un entorno accesible para la enseñanza y la experimentación, y no un sistema de producción (K. Lange, 2025).

La arquitectura de ToaruOS es híbrida. Esto significa que el kernel es en gran medida monolítico por lo que, incluye directamente los módulos significativos del sistema, como la administración de memoria, procesos, controladores de dispositivos y el sistema de archivos, pero incluye características de microkernel como la posibilidad

de ejecución modular. Este diseño posibilita la integración y el rendimiento en un entorno práctico. El lenguaje de implementación es C, con cantidades inferiores en ensamblador para interrelaciones de bajo nivel con el hardware y el arranque (K. Lange, 2025).

Los elementos nucleares del sistema comprenden los elementos básicos de cualquier sistema operativo contemporáneo. Para la gestión de procesos pone en práctica tareas múltiples planificadas basada en prioridades, de igual manera de resistir hilos ligeros. Para la gestión de memoria abarca paginación, segmentación y asignación dinámica de memoria para los programas de usuario. Al respecto del sistema de archivos, dispone de su peculiar implementación denominada “tmpfs” para sistemas en memoria, asimismo la compatibilidad con EXT2 en versiones más actuales. Por otra parte, cuenta con un sistema gráfico completo, compuesto por un servidor de ventanas (Toaru Window Server), un gestor de ventanas y librerías gráficas que permiten ejecutar aplicaciones con interfaces gráficas, como un editor de texto o un navegador web básico (K. Lange, 2025).

La comunidad de ToaruOS es reducida en comparación con otros proyectos de código abierto, pero se mantiene activa en torno a su repositorio en GitHub, donde se documentan los cambios y se comparten implementaciones experimentales. La documentación disponible proviene principalmente de su creador, quien mantiene guías, el repositorio en Github y un blog explicativo sobre su evolución. Se ha mencionado grosso modo en limitadas ocasiones en artículos científicos de especialidad.

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Tabla 2.1: Comparación resumida de distintos sistemas operativos en orden de popularidad, elaborada en base a la investigación realizada en este proyecto.

Sistema	Arquitectura	Lenguaje	Kernel	HW	Documentación	Comunidad
Windows	Monolítica	C++	Grande	Amplio	Excelente	Masiva
XNU	Híbrida	C++	Grande	Intel	Buena	Apple
Linux	Monolítica	C	Grande	Amplio	Excelente	Activa
MINIX	Microkernel	C	Pequeño	Limitado	Buena	Académica
Redox	Microkernel	Rust	Mediano	Limitado	Moderada	Experimental
xv6	Monolítica	C	Pequeño	Limitado	Excelente	Académica
FreeRTOS	Microkernel	C	Minúsculo	Amplio	Extensa	Embebidos
Fuchsia	Microkernel	C++	Mediano	IoT	Buena	Google
Haiku	Híbrida	C++	Mediano	Limitado	Buena	Constante
ToaruOS	Híbrida	C	Pequeño	Limitado	Básica	Reducida

Referencias

Alyssa Marie Rafael Natividad (2023). Windows NT: A Comprehensive Analysis of its Role in Operating System Architecture. Preprint. Accedido el 21 de septiembre de 2025.

Anthony Serino and Liang Cheng (2019). A Survey of Real-Time Operating Systems. Technical Report LU-CSE-19-003, Lehigh University, Department of Computer Science and Engineering, Bethlehem, PA. Accedido el 20 de septiembre de 2025.

Daan Keuper (2012). XNU: a security evaluation. Master's Thesis, University of Twente. Accedido el 20 de septiembre de 2025.

Frans Kaashoek and Robert Morris (2016). Xv6, a simple Unix-like teaching operating system. Accedido: 16 de septiembre de 2025.

Fuchsia Open Source Community (2025). Community Involvement in Fuchsia. Foros y repositorios. Accedido: 20 septiembre 2025.

Fuchsia Project (2025). Fuchsia — Open Source Operating System. Sitio web oficial. Accedido: 20 septiembre 2025.

Google Fuchsia Team (2025a). Fuchsia Concepts and APIs. Documentación técnica. Accedido: 20 septiembre 2025.

Google Fuchsia Team (2025b). Zircon Kernel — Fuchsia Documentation. Documentación técnica. Accedido: 20 septiembre 2025.

Google Inc. (2024). Flutter and Dart in Fuchsia. Blog técnico de Google. Accedido: 20 septiembre 2025.

GQ Informática (2023). macOS, información completa. Accedido el 20 de septiembre de 2025.

Haiku Project (2023). OpenBFS — Be File System in Haiku. Documentación técnica. Accedido: 20 septiembre 2025.

Haiku Project (2025a). Haiku — Official Website. Sitio web oficial. Accedido: 20 septiembre 2025.

Haiku Project (2025b). Haiku — The Open Source Operating System. Sitio web oficial. Accedido: 20 septiembre 2025.

Haiku Project Community (2025). Haiku Community and Development. Foros y repositorios. Accedido: 20 septiembre 2025.

Haiku Project Developers (2024). Haiku Developer Documentation. Documentación técnica. Accedido: 20 septiembre 2025.

Jonathan Levin (2007). Inside the Mac OS X Kernel. In *Proceedings of the 24th Chaos Communication Congress (24C3)*. Accedido el 20 de septiembre de 2025.

- Jorrit N. Herder and Herbert Bos and Jakob Lichtenberg and Peter M. Chen and Andrew S. Tanenbaum (2007). The Architecture of a Reliable Operating System. Accedido: 16 de septiembre de 2024.
- Justin E. Forrester and Barton P. Miller (2000). An Empirical Study of the Robustness of Windows NT Applications Using Random Testing. In *Proceedings of the 4th USENIX Windows Systems Symposium*, Seattle, WA, USA. Accedido el 21 de septiembre de 2025.
- K. Lange (2025). ToaruOS — Experimental Operating System. Repositorio oficial en GitHub. Accedido: 21 septiembre 2025.
- Luis Llamas (2025). FreeRTOS Cheatsheet, la chuleta con lo imprescindible. Accedido el 20 de septiembre de 2025.
- Mark E. Russinovich and David A. Solomon (2005). *Microsoft Windows Internals, Fourth Edition*. Microsoft Press, 4th edition. Accedido el 21 de septiembre de 2025.
- Nannan He and Han-Way Huang (2020). USE OF FreeRTOS IN TEACHING A REAL-TIME EMBEDDED SYSTEMS DESIGN COURSE. *Computers in Education Journal*, XI(2). Accedido el 20 de septiembre de 2025.
- Oracle latam (2024). ¿Qué es Linux? Accedido: 16 de septiembre de 2024.
- Richard Barry (2018). *Mastering the FreeRTOS Real Time Kernel - A Hands-On Tutorial Guide*. Real Time Engineers Ltd. For FreeRTOS V10.0.0. Accedido el 20 de septiembre de 2025.

- Robert Love (2010). *Linux Kernel Development*. Addison-Wesley Professional, Third edition.
- Rohit Saini (2018). Priority Based Scheduler in Rust Based Redox Operating System. Master's thesis, Indian Institute of Technology Madras. Accedido el 20 de septiembre de 2025.
- Russ Cox and Frans Kaashoek and Robert Morris (2022). xv6: a simple, Unix-like teaching operating system. Accedido: 17 de septiembre de 2025.
- Simon Ellmann and Paul Emmerich (2019). Porting ixy.rs to Redox. Technical report, Technical University of Munich. Accedido el 20 de septiembre de 2025.
- The FreeRTOS Project (2025). FreeRTOS - Market leading RTOS for embedded systems. Accedido el 20 de septiembre de 2025.
- The Redox OS Community (2025). The Redox OS Book. Accedido el 20 de septiembre de 2025.
- ToaruOS Project (2025). Logo de ToaruOS. Imagen PNG en sitio oficial. Accedido: 21 septiembre 2025. Licencia NCSA / University of Illinois.
- Tomáš Ritter (2019). Disk Encryption in Redox OS. Master's thesis, Masaryk University, Faculty of Informatics. Accedido el 20 de septiembre de 2025.
- Usenix (2010). MINIX 3: status report and current research. Accedido: 16 de septiembre de 2024.

Wikimedia Commons contributors (2021). Logotipo y marca denominativa de Windows - 2021. Archivo SVG en Wikipedia. Subido el 17 de octubre de 2021. Basado en el logotipo oficial de Microsoft Windows 11.

Wikipedia (2022). Nucleo linux. Accedido: 17 de septiembre de 2024.

Wikipedia contributors (2025). Redox (operating system) — Wikipedia, The Free Encyclopedia. [Online; accessed 20-September-2025].